cam_jpeg_top.v, explained part by part

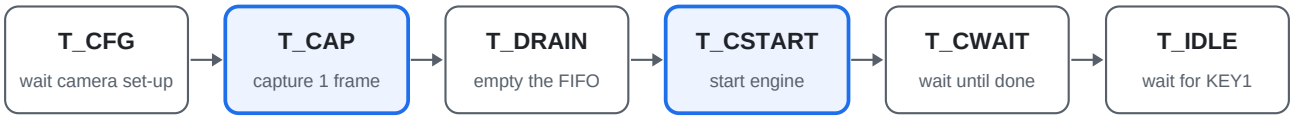
Top-level module of the live camera JPEG compression project on the DE2-115. Prepared by Dev Vijay for the project review.

1. What the module does

cam_jpeg_top is the **top-level entity**: its ports are the real pins of the board (clock, keys, switches, LEDs, 7-segment displays, VGA and camera). It does four jobs:

- **Connects the sub-modules:** cam_config, cam_capture, cam_async_fifo, color_dct_engine and color_saved_display.
- **Controls the sequence:** set up the camera, capture a frame, wait for the FIFO to empty, compress, then stop (snapshot) or repeat (live mode).
- **Holds the memories:** the original picture (RGB565) and the three result memories (Y, Cb, Cr).
- **Drives the VGA monitor:** 640 x 480 at 60 Hz, original on the left, compressed on the right.

Control sequence (state machine ctl)



SW17 = 1 (live): T_CWAIT goes straight back to T_CAP; SW17 = 0: it stops in T_IDLE until KEY1

Memory used inside the FPGA

Memory	Size	Bits	Written by	Read by
orig_mem	102,400 x 16 (RGB565)	1,638,400	camera (through FIFO)	engine and VGA left
res_y	102,400 x 8	819,200	engine, Y pass	VGA right
res_cb	25,600 x 8 (160 x 160)	204,800	engine, Cb pass	VGA right
res_cr	25,600 x 8 (160 x 160)	204,800	engine, Cr pass	VGA right
Total		2,867,200	about 350 of the 432 M9K memory blocks	

2. The code, part by part

Part A - Ports: the board pins (lines 22-49)

Lines 22-49

```
module cam_jpeg_top (
    input  wire        CLOCK_50,
    input  wire [1:0]  KEY,
    input  wire [17:0] SW,

    output reg  [7:0]  VGA_R,
    output reg  [7:0]  VGA_G,
    output reg  [7:0]  VGA_B,
    output reg         VGA_HS,
    output reg         VGA_VS,
    output wire        VGA_CLK,
    output reg         VGA_BLANK_N,
    output wire        VGA_SYNC_N,

    output wire [2:0]  LEDG,
    output wire [6:0]  HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, HEX6, HEX7,

    // TRDB-D5M camera (on the GPIO connector)
    input  wire [11:0] D5M_D,
    input  wire        D5M_FVAL,
    input  wire        D5M_LVAL,
    input  wire        D5M_PIXLCK,
    output wire        D5M_RESET_N,
    output wire        D5M_SCLK,
    inout  wire        D5M_SDATA,
    output wire        D5M_TRIGGER,
    output wire        D5M_XCLKIN
);
```

- Each port name matches a line in **cam_pins.tcl**, which ties it to a physical FPGA pin.
- VGA_R/G/B are 8 bits each because the DE2-115 has an 8-bit-per-colour video DAC (ADV7123).
- Camera ports start with D5M_; **D5M_SDATA is inout** because I2C data goes both ways.

Part B - Reset, 25 MHz clock, camera clock (lines 51-58)

Lines 51-58

```
    wire reset = ~KEY[0];

    // ---------- 25 MHz clock (VGA, engine, and the camera's input clock) ----------
    reg clk25 = 1'b0;
    always @(posedge CLOCK_50) clk25 <= ~clk25;

    assign D5M_XCLKIN  = clk25;
    assign D5M_TRIGGER = 1'b1;
```

- **reset = ~KEY[0]**: the keys are active low (0 when pressed), so it is inverted.
- **clk25** toggles on every 50 MHz edge, giving 25 MHz: the VGA pixel clock for 640 x 480 at 60 Hz. The engine and memories use it too, so most of the design is on one clock.
- **D5M_XCLKIN = clk25** is the camera's input clock; **D5M_TRIGGER = 1** keeps the camera in normal running mode.

Part C - The three camera blocks (lines 60-108)

cam_config and cam_capture instances, lines 60-88

```
    // ---------- 1. Camera set-up over I2C ----------
    wire cfg_done;
    cam_config config_unit (
        .clk          (clk25),
        .reset        (reset),
        .exposure_sel (SW[16:13]),
        .cam_reset_n  (D5M_RESET_N),
        .sclk         (D5M_SCLK),
        .sdata        (D5M_SDATA),
        .done         (cfg_done)
    );

    // ---------- 2. Capture one frame (camera clock domain) ----------
    reg         cap_req = 1'b0;
    wire        cap_done;
    wire        cam_wen;
    wire [32:0] cam_wdata;

    cam_capture capture_unit (
        .pclk        (D5M_PIXLCK),
        .d           (D5M_D),
        .lval        (D5M_LVAL),
        .fval        (D5M_FVAL),
        .bayer_phase (SW[3:2]),
        .cap_req     (cap_req),
        .cap_done    (cap_done),
        .wen         (cam_wen),
        .wdata       (cam_wdata)
    );
```

- **cam_config** gets the exposure switches SW16-SW13 and drives reset, SCLK and SDATA. Its **done** becomes cfg_done.
- **cam_capture** runs on **D5M_PIXLCK**, the camera's clock, and gets SW3-SW2 for the colour order.
- **cap_req** is a register here (controlled by the state machine); cap_done comes back from cam_capture.

FIFO instance, lines 90-108

```
    // ---------- 3. Cross from camera clock to 25 MHz ----------
    wire [32:0] fifo_q;
    wire        fifo_empty;
    wire        fifo_full;

    cam_async_fifo #(.W(33), .AW(4)) fifo_unit (
        .wclk   (D5M_PIXLCK),
        .wen    (cam_wen),
        .wdata  (cam_wdata),
        .wfull  (fifo_full),
        .rclk   (clk25),
        .ren    (1'b1),                // always take a pixel when one is there
        .rdata  (fifo_q),
        .rempty (fifo_empty)
    );

    wire        pix_we   = ~fifo_empty;
    wire [16:0] pix_addr = fifo_q[32:16];
    wire [15:0] pix_rgb  = fifo_q[15:0];
```

- The FIFO writes on the camera clock and reads on clk25. **ren = 1**: take a pixel whenever one is waiting.
- **pix_we = ~fifo_empty**: a pixel is ready to be written into orig_mem this clock.
- fifo_q is split back into its address (bits 32-16) and RGB565 pixel (bits 15-0).

Part D - Control state machine (lines 110-174)

Registers and reset, lines 110-131

```
    // ---------- 4. Control: capture -> compress -> (repeat in live mode) ----------
    localparam T_CFG = 3'd0, T_CAP = 3'd1, T_DRAIN = 3'd2,
               T_CSTART = 3'd3, T_CWAIT = 3'd4, T_IDLE = 3'd5;

    reg [2:0] ctl = T_CFG;
    reg [1:0] capdone_s = 2'b00;      // cap_done synchronised to 25 MHz
    reg [2:0] key1_s = 3'b000;        // KEY1 synchronised, for edge detection
    reg [3:0] dly = 4'd0;
    reg       eng_kick = 1'b0;        // 1-clock pulse: start the engine
    reg       first = 1'b1;           // no picture compressed yet
    wire      eng_done;

    always @(posedge clk25) begin
        capdone_s <= {capdone_s[0], cap_done};
        key1_s    <= {key1_s[1:0], ~KEY[1]};
        eng_kick  <= 1'b0;

        if (reset) begin
            ctl     <= T_CFG;
            cap_req <= 1'b0;
            first   <= 1'b1;
        end else begin
```

- **capdone_s**: cap_done comes from the camera clock, so it passes through 2 flip-flops (a synchroniser).
- **key1_s**: KEY1 through 3 flip-flops; comparing the last two finds the moment the key is pressed (edge detection), so one press gives exactly one picture.
- **eng_kick**: a one-clock pulse that restarts the engine. **first** = 1 until the first picture is compressed, so the display does not show a result for an empty memory.

States and LEDs, lines 132-174

```
            case (ctl)
            T_CFG:                                      // wait for camera set-up
                if (cfg_done) ctl <= T_CAP;

            T_CAP: begin                                // ask for one frame
                cap_req <= 1'b1;
                if (capdone_s[1]) begin
                    cap_req <= 1'b0;
                    dly     <= 4'd0;
                    ctl     <= T_DRAIN;
                end
            end

            T_DRAIN:                                    // let the FIFO empty out
                if (!capdone_s[1] && fifo_empty) begin
                    dly <= dly + 4'd1;
                    if (dly == 4'd15) ctl <= T_CSTART;
                end else
                    dly <= 4'd0;

            T_CSTART: begin                             // start compression
                eng_kick <= 1'b1;
                first    <= 1'b0;
                dly      <= 4'd0;
                ctl      <= T_CWAIT;
            end

            T_CWAIT: begin                              // wait until compression is done
                if (dly != 4'd15) dly <= dly + 4'd1;
                else if (eng_done) ctl <= SW[17] ? T_CAP : T_IDLE;
            end

            T_IDLE:                                     // wait for KEY1
                if (key1_s[1] && !key1_s[2]) ctl <= T_CAP;

            default: ctl <= T_CFG;
            endcase
        end
    end

    assign LEDG[0] = eng_done & ~first;
    assign LEDG[1] = cfg_done;
    assign LEDG[2] = (ctl == T_CAP);
```

State	What happens	Leaves when
T_CFG	waits for the camera set-up	cfg_done = 1
T_CAP	cap_req = 1: asks cam_capture for one whole frame (LEDG2 on)	cap_done arrives -> cap_req = 0
T_DRAIN	waits for the handshake to finish and the FIFO to be empty for 16 clocks	all pixels are in orig_mem
T_CSTART	eng_kick = 1 for one clock, first = 0	next clock
T_CWAIT	waits 15 clocks (so the old done flag clears), then waits for eng_done	SW17 = 1 -> T_CAP, else T_IDLE
T_IDLE	waits for a KEY1 press	KEY1 pressed -> T_CAP

LEDG0 = compression finished, LEDG1 = camera set up, LEDG2 = capturing.

Part E - VGA timing and screen addresses (lines 176-204)

Lines 176-204

```
    // ---------- VGA 640x480 timing ----------
    localparam H_VISIBLE = 640, H_FRONT = 16, H_SYNC = 96, H_BACK = 48;
    localparam V_VISIBLE = 480, V_FRONT = 10, V_SYNC = 2,  V_BACK = 33;
    localparam H_TOTAL = H_VISIBLE + H_FRONT + H_SYNC + H_BACK;
    localparam V_TOTAL = V_VISIBLE + V_FRONT + V_SYNC + V_BACK;

    reg [9:0] x = 0, y = 0;
    always @(posedge clk25) begin
        if (x == H_TOTAL - 1) begin
            x <= 0;
            y <= (y == V_TOTAL - 1) ? 10'd0 : y + 10'd1;
        end else
            x <= x + 10'd1;
    end

    wire hs_now      = ~((x >= H_VISIBLE + H_FRONT) && (x < H_VISIBLE + H_FRONT + H_SYNC));
    wire vs_now      = ~((y >= V_VISIBLE + V_FRONT) && (y < V_VISIBLE + V_FRONT + V_SYNC));
    wire visible_now = (x < H_VISIBLE) && (y < V_VISIBLE);
    wire left_now    = (x < 320) && (y < 320);
    wire right_now   = (x >= 320) && (x < 640) && (y < 320);

    wire [9:0]  x_right    = x - 10'd320;
    wire [16:0] row_base   = ({7'd0, y} << 8) + ({7'd0, y} << 6);       // y * 320
    wire [16:0] addr_left  = left_now  ? row_base + {7'd0, x}       : 17'd0;
    wire [16:0] addr_right = right_now ? row_base + {7'd0, x_right} : 17'd0;

    wire [8:0]  cy     = y[9:1];
    wire [8:0]  cx     = x_right[9:1];
    wire [16:0] addr_c = right_now ? ({8'd0, cy} << 7) + ({8'd0, cy} << 5) + {8'd0, cx} : 17'd0;
```

	Visible	Front porch	Sync pulse	Back porch	Total
Horizontal (pixels)	640	16	96	48	800
Vertical (lines)	480	10	2	33	525

- **x, y** count the position on the screen, including the invisible porch and sync parts. 25 MHz / (800 x 525) = **59.5 frames per second**.
- **hs_now, vs_now** are the sync pulses, active low (0 during the sync part).
- **left_now**: x < 320 and y < 320; **right_now**: 320 <= x < 640 and y < 320. Below row 320 the screen is black.
- **row_base = y x 320** using shifts: (y << 8) + (y << 6). addr_left and addr_right are the pixel addresses for each half.
- **addr_c**: Cb and Cr are half size, so the colour address uses x/2 and y/2 in a 160-wide image: (cy << 7) + (cy << 5) = cy x 160.

Part F - Original picture memory (lines 206-220)

Lines 206-220

```
    // ---------- Original picture memory (RGB565) ----------
    //   port A : camera writes (capture) / engine reads (compress)
    //   port B : VGA reads
    reg [15:0] orig_mem [0:102399];

    wire [16:0] eng_rd_addr;
    wire [16:0] a_addr = pix_we ? pix_addr : eng_rd_addr;
    reg  [15:0] orig_eng, orig_vga;

    always @(posedge clk25) begin
        if (pix_we) orig_mem[a_addr] <= pix_rgb;
        orig_eng <= orig_mem[a_addr];
    end
    always @(posedge clk25)
        orig_vga <= orig_mem[addr_left];
```

- A **true dual-port** block RAM: 102,400 x 16 bits.
- **Port A** is shared in time: while capturing, pixels from the FIFO are written (a_addr = pix_addr); while compressing, the engine reads (a_addr = eng_rd_addr). The state machine makes sure these never overlap.
- **Port B** is read by the VGA every clock for the left half.
- Reads are clocked, so data appears **one clock after** the address (handled in Part J).

Part G - Result memories (lines 222-244)

Lines 222-244

```
    // ---------- Result memories (engine writes, VGA reads) ----------
    reg [7:0] res_y  [0:102399];
    reg [7:0] res_cb [0:25599];
    reg [7:0] res_cr [0:25599];

    wire [16:0] eng_wr_addr;
    wire [7:0]  eng_wr_data;
    wire        eng_we;
    wire [1:0]  eng_wr_chan;

    reg [7:0] y_vga, cb_vga, cr_vga;
    always @(posedge clk25) begin
        if (eng_we && eng_wr_chan == 2'd0) res_y[eng_wr_addr] <= eng_wr_data;
        y_vga <= res_y[addr_right];
    end
    always @(posedge clk25) begin
        if (eng_we && eng_wr_chan == 2'd1) res_cb[eng_wr_addr[14:0]] <= eng_wr_data;
        cb_vga <= res_cb[addr_c[14:0]];
    end
    always @(posedge clk25) begin
        if (eng_we && eng_wr_chan == 2'd2) res_cr[eng_wr_addr[14:0]] <= eng_wr_data;
        cr_vga <= res_cr[addr_c[14:0]];
    end
```

- Three separate memories: Y full size, Cb and Cr half size. **eng_wr_chan** selects which one the engine writes.
- Cb and Cr have 25,600 entries, so only the lower 15 address bits are used ([14:0]).
- Each memory is also read by the VGA for the right half (y_vga, cb_vga, cr_vga).

Part H - Engine and display instances (lines 246-268)

Lines 246-268

```
    // ---------- Compression engine (same as the colour project) ----------
    wire [23:0] total_bits;

    color_dct_engine engine (
        .clk        (clk25),
        .reset      (reset | eng_kick | first),
        .quality    (SW[1:0]),
        .rd_addr    (eng_rd_addr),
        .rd_rgb     (orig_eng),
        .wr_addr    (eng_wr_addr),
        .wr_data    (eng_wr_data),
        .we         (eng_we),
        .wr_chan    (eng_wr_chan),
        .done       (eng_done),
        .total_bits (total_bits)
    );

    color_saved_display display (
        .done       (eng_done & ~first),
        .total_bits (total_bits),
        .HEX0 (HEX0), .HEX1 (HEX1), .HEX2 (HEX2), .HEX3 (HEX3),
        .HEX4 (HEX4), .HEX5 (HEX5), .HEX6 (HEX6), .HEX7 (HEX7)
    );
```

- **reset | eng_kick | first**: the engine restarts on KEY0, on each start pulse, and is held stopped until the first picture is captured.
- **quality = SW[1:0]** sets the compression strength.
- The engine is **exactly the same file** as in the colour project; only its source of pixels changed.
- The display shows the % only when **eng_done** and not first.

Part I - Colours for the screen (lines 270-287)

Lines 270-287

```
    // ---------- Left side: RGB565 -> 8 bits per colour ----------
    wire [7:0] o_r = {orig_vga[15:11], orig_vga[15:13]};
    wire [7:0] o_g = {orig_vga[10:5],  orig_vga[10:9]};
    wire [7:0] o_b = {orig_vga[4:0],   orig_vga[4:2]};

    // ---------- Right side: Y Cb Cr -> R G B ----------
    wire signed [17:0] yv  = {10'd0, y_vga};
    wire signed [17:0] cbd = $signed({10'd0, cb_vga}) - 18'sd128;
    wire signed [17:0] crd = $signed({10'd0, cr_vga}) - 18'sd128;

    wire signed [17:0] r_t = yv + ((18'sd359 * crd) >>> 8);
    wire signed [17:0] g_t = yv - ((18'sd88 * cbd + 18'sd183 * crd) >>> 8);
    wire signed [17:0] b_t = yv + ((18'sd454 * cbd) >>> 8);

    function [7:0] clip;
        input signed [17:0] v;
        clip = (v < 0) ? 8'd0 : (v > 255) ? 8'd255 : v[7:0];
    endfunction
```

- **Left half**: RGB565 back to 8 bits per colour by repeating the top bits (e.g. 5-bit red r becomes {r, r[4:2]}).
- **Right half**: Y, Cb, Cr back to R, G, B with the JPEG inverse formulas, multiplied by 256 to use whole numbers:

```
R = Y + 1.402 (Cr-128)                    -> 359 / 256 = 1.402
G = Y - 0.344 (Cb-128) - 0.714 (Cr-128)   ->  88 / 256 = 0.344,  183 / 256 = 0.715
B = Y + 1.772 (Cb-128)                    -> 454 / 256 = 1.773
```

- **>>> 8** divides by 256 keeping the sign.
- **clip** limits each colour to 0..255, because the formulas can go slightly below 0 or above 255.

Part J - One-clock delay and the final output (lines 289-313)

Lines 289-313

```
    // ---------- Delay control signals 1 clock (memory latency) ----------
    reg left_d, right_d, visible_d;
    always @(posedge clk25) begin
        VGA_HS      <= hs_now;
        VGA_VS      <= vs_now;
        VGA_BLANK_N <= visible_now;
        visible_d   <= visible_now;
        left_d      <= left_now;
        right_d     <= right_now;
    end

    always @(*) begin
        if (visible_d && left_d) begin
            VGA_R = o_r;  VGA_G = o_g;  VGA_B = o_b;
        end else if (visible_d && right_d) begin
            VGA_R = clip(r_t);  VGA_G = clip(g_t);  VGA_B = clip(b_t);
        end else begin
            VGA_R = 8'd0; VGA_G = 8'd0; VGA_B = 8'd0;
        end
    end

    assign VGA_CLK    = clk25;
    assign VGA_SYNC_N = 1'b0;

endmodule
```

- Memories answer one clock after the address is given. The sync signals and the left/right flags are therefore **delayed by one clock** (left_d, right_d, visible_d) so they line up with the pixel data. Without this, the picture would be shifted by one pixel.
- The final **always @(*)** picks the original colour on the left, the decompressed colour on the right, and black elsewhere.
- **VGA_CLK = clk25** tells the video DAC when to sample; **VGA_SYNC_N = 0** because sync-on-green is not used.

3. Questions the professor may ask

Q: Why 25 MHz?

It is the pixel clock of the 640 x 480, 60 Hz VGA standard (exactly 25.175 MHz; 25 MHz works on all monitors). Using the same clock for the engine avoids extra clock crossings.

Q: How do the camera and the engine share one memory port?

In time: the state machine captures first and compresses only after the FIFO is empty, so writes and engine reads never happen together.

Q: Why is there a T_DRAIN state?

After the camera finishes a frame, a few pixels can still be in the FIFO. Starting the engine before they are written would compress an incomplete picture.

Q: How does live mode work?

With SW17 = 1, T_CWAIT goes straight back to T_CAP after each compression, so the system keeps capturing and compressing, about 3-4 pictures per second.

Q: Why are some signals delayed by one clock?

Block RAM output comes one clock after the address. Delaying sync and area flags by the same amount keeps every pixel in the right place.

Q: How much FPGA memory does it use?

About 2.87 Mbit, roughly 350 of the 432 M9K blocks on the Cyclone IV EP4CE115.

Q: What do the LEDs mean?

LEDG1 camera set up, LEDG2 capturing, LEDG0 compression finished.